

Casos de Teste

Banking API v1.0

Projeto: Banking API

Versão: 1.0.0

Preparado por: [Danielle Monteiro]

Data: 19/02/2026

Total de Casos de Teste: 30

Módulos Cobertos: Backend (API) + Frontend Web + Frontend Mobile

Resumo dos Casos de Teste

ID	Módulo	Funcionalidade	Tipo	Prioridade
TC-001	Backend	Criar Conta	Positivo	Alta
TC-002	Backend	Criar Conta	Negativo	Alta
TC-003	Backend	Criar Conta	Negativo	Alta
TC-004	Backend	Criar Conta	Negativo	Média
TC-005	Backend	Listar Contas	Positivo	Alta
TC-006	Backend	Listar Contas	Positivo	Alta
TC-007	Backend	Buscar por ID	Positivo	Alta
TC-008	Backend	Buscar por ID	Negativo	Alta
TC-009	Backend	Atualizar Conta	Positivo	Média
TC-010	Backend	Deletar Conta	Positivo	Alta
TC-011	Backend	Deletar Conta	Negativo	Alta
TC-012	Backend	Depósito	Positivo	Alta
TC-013	Backend	Depósito	Negativo	Alta
TC-014	Backend	Depósito	Negativo	Alta
TC-015	Backend	Transferência	Positivo	Alta
TC-016	Backend	Transferência	Negativo	Alta
TC-017	Backend	Transferência	Negativo	Alta
TC-018	Backend	Transferência	Negativo	Alta
TC-019	Frontend Web	Criar Conta - Formulário	Positivo	Alta
TC-020	Frontend Web	Criar Conta - Validação UI	Negativo	Alta
TC-021	Frontend Web	Listar Contas - Tabela	Positivo	Alta
TC-022	Frontend Web	Depósito - Desktop	Positivo	Alta
TC-023	Frontend Web	Transferência - Desktop	Positivo	Alta
TC-024	Frontend Web	Transferência - Saldo insuf.	Negativo	Alta
TC-025	Frontend Web	Deletar Conta - Desktop	Positivo	Média
TC-026	Frontend Mobile	Criar Conta - Mobile	Positivo	Alta
TC-027	Frontend Mobile	Saldo Total - Home	Positivo	Alta
TC-028	Frontend Mobile	Depósito - Mobile	Positivo	Alta
TC-029	Frontend Mobile	Transferência - Mobile	Positivo	Alta
TC-030	Frontend Mobile	Navegação por Abas	Positivo	Média

Seção 1 — Backend (API REST)

Os testes de backend são executados diretamente nos endpoints da API via Swagger UI (<http://localhost:3000/api-docs>) ou Postman. Não é necessário interagir com nenhum frontend.

TC-001 — Criar Conta Backend			
Módulo: Backend	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Criar conta com todos os dados válidos e saldo inicial positivo

Pré-condições:

- API rodando em <http://localhost:3000>
- Nenhuma conta com CPF 111.111.111-11 cadastrada

Dados de Entrada:

- name: "João Silva"
- email: "joao@email.com"
- cpf: "111.111.111-11"
- initialBalance: 500

Passos de Execução:

1. Acessar POST /accounts no Swagger UI ou Postman
2. Inserir no body: {"name": "João Silva", "email": "joao@email.com", "cpf": "111.111.111-11", "initialBalance": 500}
3. Clicar em Execute / Send
4. Verificar o código de resposta HTTP e o body retornado

Resultado Esperado:

- Status HTTP: 201 Created
- Body contém: id (UUID gerado), name, email, cpf, balance: 500, createdAt
- Conta pode ser localizada via GET /accounts

Resultado Obtido: _____

TC-002 — Criar Conta Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Tentar criar conta sem o campo obrigatório 'name'

Pré-condições:

- API rodando em <http://localhost:3000>

Dados de Entrada:

- name: {ausente}
- email: "teste@email.com"
- cpf: "222.222.222-22"
- initialBalance: 100

Passos de Execução:

5. Acessar POST /accounts no Swagger UI ou Postman
6. Inserir no body: {"email": "teste@email.com", "cpf": "222.222.222-22", "initialBalance": 100}
7. Executar a requisição
8. Observar o código HTTP e a mensagem retornada

Resultado Esperado:

- Status HTTP: 400 Bad Request
- Body contém: {"error": "Campos obrigatórios: name, email, cpf, initialBalance"}
- Nenhuma conta criada no sistema

Resultado Obtido: _____

TC-003 — Criar Conta Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Tentar criar conta com CPF já cadastrado no sistema

Pré-condições:

- API rodando
- Já existe uma conta com CPF 111.111.111-11 cadastrada (executar TC-001 antes)

Dados de Entrada:

- name: "Carlos Souza"
- email: "carlos@email.com"
- cpf: "111.111.111-11" ← CPF duplicado
- initialBalance: 200

Passos de Execução:

9. Garantir que TC-001 foi executado com sucesso
10. Acessar POST /accounts
11. Inserir no body: {"name": "Carlos Souza", "email": "carlos@email.com", "cpf": "111.111.111-11", "initialBalance": 200}
12. Executar a requisição

Resultado Esperado:

- Status HTTP: 400 Bad Request
- Body contém: {"error": "CPF já cadastrado"}
- Nenhuma nova conta criada

Resultado Obtido: _____

TC-004 — Criar Conta Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Média	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Tentar criar conta com saldo inicial negativo

Pré-condições:

- API rodando

Dados de Entrada:

- name: "Ana Lima"
- email: "ana@email.com"
- cpf: "333.333.333-33"
- initialBalance: -100 ← valor negativo

Passos de Execução:

13. Acessar POST /accounts

14. Inserir no body: {"name": "Ana Lima", "email": "ana@email.com", "cpf": "333.333.333-33", "initialBalance": -100}

15. Executar a requisição

Resultado Esperado:

- Status HTTP: 400 Bad Request
- Body contém: {"error": "initialBalance deve ser um número >= 0"}
- Nenhuma conta criada

Resultado Obtido: _____

TC-005 — Listar Contas Backend			
Módulo: Backend	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Listar contas quando não há nenhuma conta cadastrada

Pré-condições:

- API rodando
- Nenhuma conta cadastrada (banco limpo / servidor recém iniciado)

Passos de Execução:

16. Acessar GET /accounts

17. Executar a requisição sem parâmetros

Resultado Esperado:

- Status HTTP: 200 OK
- Body retorna: [] (array vazio)
- Nenhum dado de conta exibido

Resultado Obtido: _____

TC-006 — Listar Contas Backend			
Módulo: Backend	Tipo de Teste: Positivo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Listar contas quando existem contas cadastradas

Pré-condições:

- API rodando
- Pelo menos 2 contas cadastradas (executar TC-001 e criar mais uma)

Passos de Execução:

18. Garantir que ao menos 2 contas foram criadas
19. Acessar GET /accounts
20. Executar a requisição

Resultado Esperado:

- Status HTTP: 200 OK
- Body retorna array com todas as contas
- Cada objeto contém: id, name, email, cpf, balance, createdAt

Resultado Obtido: _____

TC-007 — Buscar por ID Backend			
Módulo: Backend	Tipo de Teste: Positivo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Buscar conta existente usando ID válido

Pré-condições:

- API rodando
- Conta com ID conhecido cadastrada

Dados de Entrada:

- id: {id gerado no TC-001}

Passos de Execução:

21. Copiar o ID da conta criada no TC-001
22. Acessar GET /accounts/{id}

23. Inserir o ID no campo de parâmetro

24. Executar a requisição

Resultado Esperado:

- Status HTTP: 200 OK
- Body retorna os dados completos da conta
- Os dados batem com os inseridos na criação

Resultado Obtido: _____

TC-008 — Buscar por ID Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Tentar buscar conta com ID que não existe

Pré-condições:

- API rodando

Dados de Entrada:

- id: 00000000-0000-0000-0000-000000000000 ← ID inexistente

Passos de Execução:

25. Acessar GET /accounts/{id}

26. Inserir um UUID inválido ou inexistente

27. Executar a requisição

Resultado Esperado:

- Status HTTP: 404 Not Found
- Body contém: {"error": "Conta não encontrada"}

Resultado Obtido: _____

TC-009 — Atualizar Conta Backend			
Módulo: Backend	Tipo de Teste: Positivo	Prioridade: Média	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Atualizar nome e email de uma conta existente

Pré-condições:

- API rodando
- Conta existente com ID conhecido

Dados de Entrada:

- id: {id gerado no TC-001}
- name: "João Silva Atualizado"
- email: "joao.novo@email.com"

Passos de Execução:

28. Acessar PUT /accounts/{id}
29. Inserir o ID no parâmetro de rota
30. Inserir no body: {"name": "João Silva Atualizado", "email": "joao.novo@email.com"}
31. Executar a requisição

Resultado Esperado:

- Status HTTP: 200 OK
- Body retorna a conta com os novos dados
- Campos name e email estão atualizados
- CPF e balance permanecem inalterados

Resultado Obtido: _____

TC-010 — Deletar Conta Backend			
Módulo: Backend	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Deletar conta existente com ID válido

Pré-condições:

- API rodando
- Conta com ID conhecido cadastrada

Dados de Entrada:

- id: {id de uma conta existente}

Passos de Execução:

32. Acessar DELETE /accounts/{id}
33. Inserir o ID de uma conta existente
34. Executar a requisição
35. Confirmar via GET /accounts/{id} que a conta foi removida

Resultado Esperado:

- Status HTTP: 200 OK
- Body contém: {"message": "Conta removida com sucesso"}
- GET /accounts/{id} retorna 404 após a remoção

Resultado Obtido: _____

TC-011 — Deletar Conta Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Tentar deletar conta com ID inexistente

Pré-condições:

- API rodando

Dados de Entrada:

- id: 00000000-0000-0000-0000-000000000000 ← ID inexistente

Passos de Execução:

36. Acessar DELETE /accounts/{id}

37. Inserir um UUID inexistente

38. Executar a requisição

Resultado Esperado:

- Status HTTP: 404 Not Found
- Body contém: {"error": "Conta não encontrada"}

Resultado Obtido: _____

TC-012 — Depósito Backend			
Módulo: Backend	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Realizar depósito com valor positivo válido

Pré-condições:

- API rodando
- Conta com saldo R\$ 500,00 cadastrada (TC-001)

Dados de Entrada:

- id: {id da conta TC-001}
- amount: 200

Passos de Execução:

39. Acessar POST /accounts/{id}/deposit

40. Inserir o ID da conta no parâmetro de rota

41. Inserir no body: {"amount": 200}

42. Executar a requisição

Resultado Esperado:

- Status HTTP: 200 OK
- Body contém: {"message": "Depósito realizado", "balance": 700}
- Saldo anterior era 500 → novo saldo = 700

Resultado Obtido: _____

TC-013 — Depósito Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Tentar realizar depósito com valor igual a zero

Pré-condições:

- API rodando
- Conta cadastrada

Dados de Entrada:

- amount: 0

Passos de Execução:

43. Acessar POST /accounts/{id}/deposit

44. Inserir no body: {"amount": 0}

45. Executar a requisição

Resultado Esperado:

- Status HTTP: 400 Bad Request
- Body contém: {"error": "amount deve ser maior que 0"}
- Saldo da conta permanece inalterado

Resultado Obtido: _____

TC-014 — Depósito Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Tentar realizar depósito com valor negativo

Pré-condições:

- API rodando
- Conta cadastrada

Dados de Entrada:

- amount: -50

Passos de Execução:

46. Acessar POST /accounts/{id}/deposit

47. Inserir no body: {"amount": -50}

48. Executar a requisição

Resultado Esperado:

- Status HTTP: 400 Bad Request
- Body contém: {"error": "amount deve ser maior que 0"}
- Saldo da conta permanece inalterado

Resultado Obtido: _____

TC-015 — Transferência Backend			
Módulo: Backend	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Transferir valor com saldo suficiente para conta destino válida**Pré-condições:**

- API rodando
- Conta A com saldo R\$ 700,00 (após TC-012)
- Conta B com saldo R\$ 200,00 (criar antes)

Dados de Entrada:

- id (origem): {id Conta A}
- targetId: {id Conta B}
- amount: 300

Passos de Execução:

49. Criar Conta B se não existir
50. Acessar POST /accounts/{id-A}/transfer
51. Inserir no body: {"targetId": "{id-B}", "amount": 300}
52. Executar a requisição
53. Verificar saldos de ambas as contas

Resultado Esperado:

- Status HTTP: 200 OK
- sourceBalance: 400 (700 - 300)
- targetBalance: 500 (200 + 300)
- Saldos refletidos corretamente via GET

Resultado Obtido: _____

TC-016 — Transferência Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Tentar transferir valor maior que o saldo disponível

Pré-condições:

- API rodando
- Conta A com saldo R\$ 400,00

Dados de Entrada:

- id (origem): {id Conta A}
- targetId: {id Conta B}
- amount: 1000 ← maior que o saldo

Passos de Execução:

54. Acessar POST /accounts/{id-A}/transfer
55. Inserir no body: {"targetId": "{id-B}", "amount": 1000}
56. Executar a requisição

Resultado Esperado:

- Status HTTP: 400 Bad Request
- Body contém: {"error": "Saldo insuficiente"}
- Saldos de ambas as contas permanecem inalterados

Resultado Obtido: _____

TC-017 — Transferência Backend			
Módulo: Backend	Tipo de Teste: Negativo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Tentar transferir para conta destino inexistente

Pré-condições:

- API rodando
- Conta A existente

Dados de Entrada:

- id (origem): {id Conta A}
- targetId: 00000000-0000-0000-0000-000000000000 ← inexistente
- amount: 50

Passos de Execução:

57. Acessar POST /accounts/{id-A}/transfer
58. Inserir no body: {"targetId": "00000000-0000-0000-0000-000000000000", "amount": 50}
59. Executar a requisição

Resultado Esperado:

- Status HTTP: 404 Not Found
- Body contém: {"error": "Conta destino não encontrada"}
- Saldo da conta origem permanece inalterado

Resultado Obtido: _____

TC-018 — Transferência | Backend**Módulo:** Backend**Tipo de Teste:** Negativo**Prioridade:** Alta**Status:** ☐ PASS ☐ FAIL ☐ **BLOCKED****Cenário:** Tentar transferir de uma conta para ela mesma**Pré-condições:**

- API rodando
- Conta A existente

Dados de Entrada:

- id (origem): {id Conta A}
- targetId: {mesmo id Conta A}
- amount: 100

Passos de Execução:

60. Acessar POST /accounts/{id-A}/transfer

61. Inserir no body: {"targetId": "{id-A}", "amount": 100} ← mesmo ID

62. Executar a requisição

Resultado Esperado:

- Status HTTP: 400 Bad Request
- Body contém: {"error": "Conta origem e destino não podem ser iguais"}
- Saldo permanece inalterado

Resultado Obtido: _____

Seção 2 — Frontend Web (Desktop)

Os testes de frontend web são executados abrindo o arquivo `frontend-web/index.html` no browser. O servidor da API deve estar rodando em paralelo.

TC-019 — Criar Conta — Formulário Frontend Web			
Módulo: Frontend Web	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Preencher e submeter o formulário de criação de conta com dados válidos

Pré-condições:

- API rodando em `http://localhost:3000`
- `frontend-web/index.html` aberto no browser Chrome
- Nenhuma conta com o CPF a ser usado já cadastrada

Dados de Entrada:

- Nome: Maria Oliveira
- E-mail: `maria@email.com`
- CPF: `444.444.444-44`
- Saldo Inicial: 300

Passos de Execução:

63. Abrir `frontend-web/index.html` no Chrome
64. Localizar o card 'Criar Conta' no lado esquerdo
65. Preencher o campo Nome com 'Maria Oliveira'
66. Preencher o campo E-mail com '`maria@email.com`'
67. Preencher o campo CPF com '`444.444.444-44`'
68. Preencher o campo Saldo Inicial com '`300`'
69. Clicar no botão 'Criar Conta'
70. Observar a área 'Resposta da API' e a tabela de contas

Resultado Esperado:

- Área de resposta exibe JSON com status 201 e dados da nova conta
- A tabela 'Todas as Contas' atualiza e exibe a conta criada
- Saldo da conta aparece como R\$ 300,00 na tabela
- Nenhuma mensagem de erro é exibida

Resultado Obtido: _____

TC-020 — Criar Conta — Validação UI Frontend Web			
Módulo: Frontend Web	Tipo de Teste: Negativo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Tentar criar conta sem preencher o campo Nome e verificar resposta de erro

Pré-condições:

- API rodando
- frontend-web/index.html aberto no browser

Dados de Entrada:

- Nome: (vazio)
- E-mail: teste@email.com
- CPF: 555.555.555-55
- Saldo Inicial: 100

Passos de Execução:

71. Abrir frontend-web/index.html
72. No card 'Criar Conta', deixar o campo Nome vazio
73. Preencher E-mail, CPF e Saldo Inicial normalmente
74. Clicar em 'Criar Conta'
75. Observar a área de resposta

Resultado Esperado:

- Área de resposta exibe JSON de erro com fundo vermelho
- Mensagem: {"error": "Campos obrigatórios: name, email, cpf, initialBalance"}
- Nenhuma linha nova adicionada na tabela de contas
- Tabela de contas permanece inalterada

Resultado Obtido: _____

TC-021 — Listar Contas — Tabela Frontend Web			
Módulo: Frontend Web	Tipo de Teste: Positivo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Verificar se a tabela de contas exibe corretamente todas as contas cadastradas

Pré-condições:

- API rodando
- Pelo menos 2 contas cadastradas (TC-019 executado)
- frontend-web/index.html aberto

Passos de Execução:

76. Clicar no botão '🔄 Atualizar Lista' na seção 'Todas as Contas'
77. Observar a tabela carregada

Resultado Esperado:

- Tabela exibe todas as contas cadastradas
- Colunas visíveis: ID (abreviado), Nome, E-mail, CPF, Saldo
- Saldo formatado como 'R\$ XXX.XX'

- Cada linha corresponde a uma conta diferente

Resultado Obtido: _____

TC-022 — Depósito — Desktop Frontend Web			
Módulo: Frontend Web	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Realizar depósito via interface desktop e verificar atualização do saldo na tabela

Pré-condições:

- API rodando
- Conta existente com ID conhecido
- frontend-web/index.html aberto

Dados de Entrada:

- ID: {id de conta existente}
- Valor: 150

Passos de Execução:

78. Localizar o card 'Receber Dinheiro (Depósito)'

79. Preencher o campo ID com o ID da conta

80. Preencher o campo Valor com 150

81. Clicar em 'Depositar'

82. Observar a resposta e a tabela de contas

Resultado Esperado:

- Área de resposta exibe: Depósito realizado + novo saldo
- Tabela de contas atualiza com o saldo acrescido de R\$ 150,00
- Fundo verde da área de resposta (sem erro)

Resultado Obtido: _____

TC-023 — Transferência — Desktop Frontend Web			
Módulo: Frontend Web	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Realizar transferência entre duas contas via interface desktop

Pré-condições:

- API rodando
- Conta A e Conta B cadastradas
- Conta A com saldo suficiente (mín. R\$ 100,00)

Dados de Entrada:

- ID Conta Origem: {id Conta A}
- ID Conta Destino: {id Conta B}
- Valor: 100

Passos de Execução:

83. Localizar o card 'Enviar Dinheiro (Transferência)'
84. Preencher ID conta origem com ID da Conta A
85. Preencher ID conta destino com ID da Conta B
86. Preencher Valor com 100
87. Clicar em 'Transferir'
88. Observar resposta e tabela

Resultado Esperado:

- Resposta exhibe: Transferência realizada com sucesso
- Exibe sourceBalance e targetBalance atualizados
- Tabela reflete os novos saldos de ambas as contas
- Fundo verde na área de resposta

Resultado Obtido: _____

TC-024 — Transferência — Saldo Insuficiente Frontend Web			
Módulo: Frontend Web	Tipo de Teste: Negativo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Tentar realizar transferência com saldo insuficiente e verificar mensagem de erro na UI

Pré-condições:

- API rodando
- Conta A com saldo R\$ 50,00
- frontend-web/index.html aberto

Dados de Entrada:

- ID Conta Origem: {id Conta A}
- ID Conta Destino: {id Conta B}
- Valor: 999

Passos de Execução:

89. Localizar o card 'Enviar Dinheiro (Transferência)'
90. Preencher os campos conforme dados de entrada
91. Clicar em 'Transferir'
92. Observar a área de resposta

Resultado Esperado:

- Área de resposta exhibe: {"error": "Saldo insuficiente"} com fundo vermelho
- Saldos de ambas as contas permanecem inalterados na tabela
- Nenhuma transferência registrada

Resultado Obtido: _____

TC-025 — Deletar Conta — Desktop Frontend Web			
Módulo: Frontend Web	Tipo de Teste: Positivo	Prioridade: Média	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Deletar uma conta via interface desktop e confirmar remoção na tabela

Pré-condições:

- API rodando
- Conta existente com ID conhecido
- frontend-web/index.html aberto

Dados de Entrada:

- ID: {id de conta a deletar}

Passos de Execução:

93. Localizar o card 'Buscar / Deletar Conta'
94. Preencher o campo ID com o ID da conta
95. Clicar em 'Deletar Conta' (botão vermelho)
96. Observar a resposta
97. Observar a tabela de contas

Resultado Esperado:

- Resposta exhibe: {"message": "Conta removida com sucesso"}
- A conta desaparece da tabela 'Todas as Contas'
- Busca posterior pelo mesmo ID retorna erro 404

Resultado Obtido: _____

Seção 3 — Frontend Mobile

Os testes mobile são executados abrindo `frontend-mobile/index.html` no browser. A visualização simula uma tela de smartphone (largura 390px). Todos os testes devem ser executados com a API rodando.

TC-026 — Criar Conta — Mobile Frontend Mobile			
Módulo: Frontend Mobile	Tipo de Teste: Positivo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Criar nova conta pelo formulário mobile e verificar redirecionamento para a aba Início

Pré-condições:

- API rodando em `http://localhost:3000`
- `frontend-mobile/index.html` aberto no Chrome
- Nenhuma conta com CPF 666.666.666-66 cadastrada

Dados de Entrada:

- Nome: Pedro Costa
- E-mail: `pedro@email.com`
- CPF: 666.666.666-66
- Saldo Inicial: 800

Passos de Execução:

98. Abrir `frontend-mobile/index.html` no Chrome
99. Clicar na aba '+ Conta' na barra de navegação inferior
100. Preencher Nome com 'Pedro Costa'
101. Preencher E-mail com '`pedro@email.com`'
102. Preencher CPF com '666.666.666-66'
103. Preencher Saldo Inicial com '800'
104. Clicar em 'Criar Conta'
105. Observar a resposta exibida na tela

Resultado Esperado:

- Resposta exhibe JSON com os dados da conta criada
- Fundo da resposta sem indicação de erro
- Conta visível ao navegar para aba ' Início'

Resultado Obtido: _____

TC-027 — Saldo Total — Home Frontend Mobile			
Módulo: Frontend Mobile	Tipo de Teste: Positivo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Verificar se o card de saldo total exhibe a soma correta de todas as contas na tela Home

Pré-condições:

- API rodando
- Pelo menos 2 contas cadastradas com saldos conhecidos
- Exemplo: Conta A = R\$400, Conta B = R\$500 → Total = R\$900

Passos de Execução:

106. Abrir frontend-mobile/index.html
107. Estar na aba ' Início' (aba padrão)
108. Clicar em ' Atualizar'
109. Observar o card azul superior com 'Saldo Total em Sistema'

Resultado Esperado:

- Card exibe o valor total correto (soma de todos os saldos)
- Lista abaixo mostra cada conta com nome, email, CPF e saldo individual
- Saldo de cada conta formatado corretamente como 'R\$ XXX.XX'

Resultado Obtido: _____

TC-028 — Depósito — Mobile Frontend Mobile			
Módulo: Frontend Mobile	Tipo de Teste: Positivo	Prioridade: Alta	Status: [] PASS [] FAIL [] BLOCKED

Cenário: Realizar depósito via interface mobile e verificar atualização do saldo total na Home

Pré-condições:

- API rodando
- Conta existente com ID conhecido e saldo R\$ 500,00
- frontend-mobile/index.html aberto

Dados de Entrada:

- ID da Conta: {id da conta}
- Valor: 250

Passos de Execução:

110. Clicar na aba ' Transações'
111. Na seção 'Depósito (Receber)', preencher ID da Conta
112. Preencher Valor com 250
113. Clicar em ' Depositar'
114. Observar a resposta na tela
115. Navegar para aba ' Início' e verificar o saldo

Resultado Esperado:

- Resposta exibe: Depósito realizado + saldo atualizado
- Saldo total na Home aumenta em R\$ 250,00
- Conta específica na lista da Home mostra o novo saldo

Resultado Obtido: _____

TC-029 — Transferência — Mobile Frontend Mobile			
Módulo: Frontend Mobile	Tipo de Teste: Positivo	Prioridade: Alta	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Realizar transferência entre contas via interface mobile

Pré-condições:

- API rodando
- Conta A com saldo mínimo de R\$ 200,00
- Conta B existente
- frontend-mobile/index.html aberto

Dados de Entrada:

- ID Conta Origem: {id Conta A}
- ID Conta Destino: {id Conta B}
- Valor: 200

Passos de Execução:

116. Clicar na aba ' Transações'
117. Rolar para a seção 'Transferência (Enviar)'
118. Preencher ID Conta Origem com ID da Conta A
119. Preencher ID Conta Destino com ID da Conta B
120. Preencher Valor com 200
121. Clicar em ' Transferir'
122. Observar a resposta
123. Navegar para Home e verificar saldos

Resultado Esperado:

- Resposta exibe: Transferência realizada com sucesso
- Exibe sourceBalance e targetBalance atualizados
- Na Home, saldo da Conta A diminuiu R\$ 200,00
- Na Home, saldo da Conta B aumentou R\$ 200,00

Resultado Obtido: _____

TC-030 — Navegação por Abas Frontend Mobile			
Módulo: Frontend Mobile	Tipo de Teste: Positivo	Prioridade: Média	Status: ☐ PASS ☐ FAIL ☐ BLOCKED

Cenário: Verificar se a navegação entre todas as abas funciona corretamente

Pré-condições:

- frontend-mobile/index.html aberto no browser
- API rodando (para a aba Home carregar dados)

Passos de Execução:

124. Verificar que a aba ' Início' está ativa por padrão
125. Clicar na aba ' Conta' e verificar que o formulário de criação é exibido
126. Clicar na aba ' Transações' e verificar que as seções de depósito e transferência aparecem
127. Clicar na aba ' Buscar' e verificar que o campo de busca por ID é exibido
128. Retornar para aba ' Início' e verificar que os dados são recarregados

Resultado Esperado:

- Cada aba exibe seu conteúdo correto ao ser clicada
- A aba ativa fica destacada com sublinhado azul
- Ao retornar para Home, a lista de contas é recarregada automaticamente
- Nenhuma sobreposição de conteúdo entre abas

Resultado Obtido: _____

Folha de Assinaturas

Preparado por	Revisado por	Aprovado por
Nome: _____ Data: _____	Nome: _____ Data: _____	Nome: _____ Data: _____